OS U4 ORANGE

Building PES-VCS — A Version Control System from Scratch

REPORT

Nitin Somisetty – PES2UG24CS327

[NitinSomisetty/os-u4-orange: to do]

Phase 1:

Screenshot 1 A :

```
➜ os-u4-orange git:(main) ✗ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1 B :

```
➜ os-u4-orange git:(main) find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
➜ os-u4-orange git:(main)
```

Phase 2 :

Screenshot 2 A :

```
➜ os-u4-orange git:(main) ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2 B :

```
➜ os-u4-orange git:(main) find .pes/objects -type f | head -1 | xargs -I {} sh -c 'xxd {} | head
-20'
00000000: 626c 6f62 2032 3632 3636 0023 2042 7569  blob 26266.# Bui
00000010: 6c64 696e 6720 5045 532d 5643 5320 e280  lding PES-VCS ..
00000020: 9420 4120 5665 7273 696f 6e20 436f 6e74  . A Version Cont
00000030: 726f 6c20 5379 7374 656d 2066 726f 6d20  rol System from
00000040: 5363 7261 7463 680a 0a2a 2a4f 626a 6563  Scratch..**Objec
00000050: 7469 7665 3a2a 2a20 4275 696c 6420 6120  tive:** Build a
00000060: 6c6f 6361 6c20 7665 7273 696f 6e20 636f  local version co
00000070: 6e74 726f 6c20 7379 7374 656d 2074 6861  ntrol system tha
00000080: 7420 7472 6163 6b73 2066 696c 6520 6368  t tracks file ch
00000090: 616e 6765 732c 2073 746f 7265 7320 736e  anges, stores sn
000000a0: 6170 7368 6f74 7320 6566 6669 6369 656e  apshots efficien
000000b0: 746c 792c 2061 6e64 2073 7570 706f 7274  tly, and support
000000c0: 7320 636f 6d6d 6974 2068 6973 746f 7279  s commit history
000000d0: 2e20 4576 6572 7920 636f 6d70 6f6e 656e  . Every componen
000000e0: 7420 6d61 7073 2064 6972 6563 746c 7920  t maps directly
000000f0: 746f 206f 7065 7261 7469 6e67 2073 7973  to operating sys
00000100: 7465 6d20 616e 6420 6669 6c65 7379 7374  tem and filesyst
00000110: 656d 2063 6f6e 6365 7074 732e 0a0a 2a2a  em concepts...**
00000120: 506c 6174 666f 726d 3a2a 2a20 5562 756e  Platform:** Ubun
00000130: 7475 2032 322e 3034 0a0a 2d2d 2d0a 0a23  tu 22.04..---..#
```

Phase 3 :

Screenshot 3 A :

```
➜ os-u4-orange git:(main) ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  .gitignore
  untracked:  index.h
  untracked:  object.c
  untracked:  README.md
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  pes.h
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  tree.c
  untracked:  test_objects.c
```

Screenshot 3 B :

```
➜ os-u4-orange git:(main) ✗ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776692170 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776692170 6 file2.txt
➜ os-u4-orange git:(main) ✗
```

Phase 4 :

Screenshot 4 A

```
➜ os-u4-orange git:(main) ✗ >....
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
./pes log
Initialized empty PES repository in .pes/
Committed: 14ba556ac9bb... Initial commit
Committed: 7f0a68ce7ea0... Add world
Committed: 4c896357f8e8... Add farewell
commit 4c896357f8e8b2a44da420ec3335ebf7473ffc4592848d46fdc94e197703bf76
Author: PES User <pes@localhost>
Date:   1776692677

    Add farewell

commit 7f0a68ce7ea042a32e0f40731e3a66149ebf1694e7aa98587251c75652c5a6b5
Author: PES User <pes@localhost>
Date:   1776692677

    Add worldell

commit 14ba556ac9bbbd13d912235d63d56429d5f6f5655d24b0ffaefdc68398c8781b
Author: PES User <pes@localhost>
Date:   1776692676
```

```
commit 14ba556ac9bbbd13d912235d63d56429d5f6f5655d24b0ffaefdc68398c8781b
Author: PES User <pes@localhost>
Date:   1776692676

    Initial commit

commit 4c9340a6a859f8a3af0c1e13b8d90af96e419e8ac753acb4c112e3a3f6a2d5e6
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776692590

    Add farewellt

commit e825550c1e85a887be0b62495216b16a99ce5cf29c597a78558bf8de1af71694
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776692590

    Add world

commit 59869a90cb56d688b215ed1ebc97f268a9211179c0ba3114498f8e48c2ccd533
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776692590

    Initial committial commit259A

➜ os-u4-orange git:(main) ✗
```

Screenshot 4 B

```
➜ os-u4-orange git:(main) ✗ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/14/ba556ac9bbbd13d912235d63d56429d5f6f5655d24b0ffaefdc68398c8781b
.pes/objects/3b/c19934c2ccc37b9582dd42105ca4a1df8e197f51c99a3ef71628231437fdbf
.pes/objects/4c/896357f8e8b2a44da420ec3335ebf7473ffc4592848d46fdc94e197703bf76
.pes/objects/4c/9340a6a859f8a3af0c1e13b8d90af96e419e8ac753acb4c112e3a3f6a2d5e6
.pes/objects/59/869a90cb56d688b215ed1ebc97f268a9211179c0ba3114498f8e48c2ccd533
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7f/0a68ce7ea042a32e0f40731e3a66149ebf1694e7aa98587251c75652c5a6b5
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/e8/25550c1e85a887be0b62495216b16a99ce5cf29c597a78558bf8de1af71694
.pes/refs/heads/main
➜ os-u4-orange git:(main) ✗
```

Screenshot 4 C

```
➜ os-u4-orange git:(main) ✗ cat .pes/refs/heads/main
cat .pes/HEAD
4c896357f8e8b2a44da420ec3335ebf7473ffc4592848d46fdc94e197703bf76
ref: refs/heads/main
```

MakeTest - Integration

```
➜ os-u4-orange git:(main) ✗ export PES_AUTHOR="NitinSomisetty <PES2UG24CS327>"
make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 0855e73a48dc... Initial commit

Log after first commit:
commit 0855e73a48dcbe1b4b3b75d1b1832e83943f7fe06fb74d6b2cc2d75564a1cd73
Author: NitinSomisetty <PES2UG24CS327>
```

```
Log after first commit:
commit 0855e73a48dcbe1b4b3b75d1b1832e83943f7fe06fb74d6b2cc2d75564a1cd73
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776693114

    Initial commit


--- Second Commit ---
Committed: 75b45e840d8d... Update file.txt

--- Third Commit ---
Committed: ee374fab2f56... Add farewell

--- Full History ---
commit ee374fab2f56aad025acfa6b13a3b0a26a15bc3d4a551048bf5b23520c40bdfe
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776693115

    Add farewellver auth!

commit 75b45e840d8d092954fb7029ba25c5224d014a4c1b0b4b1f30a20d208df784b9
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776693115

    Update file.txt auth!

commit 0855e73a48dcbe1b4b3b75d1b1832e83943f7fe06fb74d6b2cc2d75564a1cd73
Author: NitinSomisetty <PES2UG24CS327>
Date:   1776693114
```

```
    Initial committial commitES2A


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
ee374fab2f56aad025acfa6b13a3b0a26a15bc3d4a551048bf5b23520c40bdfe

--- Object Store ---
Objects created:
10
.pes/objects/08/55e73a48dcbe1b4b3b75d1b1832e83943f7fe06fb74d6b2cc2d75564a1cd73
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/75/b45e840d8d092954fb7029ba25c5224d014a4c1b0b4b1f30a20d208df784b9
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/ee/374fab2f56aad025acfa6b13a3b0a26a15bc3d4a551048bf5b23520c40bdfe

=== All integration tests completed ===
```

Phase 5 :

Q 5.1

In Git (and PES), a branch is a file located in .pes/refs/heads/ that stores a commit hash. To implement pes checkout <branch>, the following changes are required:

- HEAD file: Update .pes/HEAD to point to
ref: refs/heads/<branch>
- Object store: No changes are needed since objects are immutable.

Next, the working directory must be updated to match the selected branch. This involves reading the commit hash from the branch file, loading the corresponding tree object, and rebuilding the directory structure. All files and folders are recreated based on the tree, and file contents are restored using blob objects from the object store.

This process is complex because it requires recursively rebuilding directories, correctly mapping stored objects back to files, and safely replacing existing files. It must also handle local changes carefully to avoid data loss from overwriting.

Q 5.2

When switching branches, it is important to ensure that uncommitted changes are not lost. A “dirty working directory” occurs when files have been modified but not committed.

To detect this, the working directory is compared with the index and the object store. The index represents the last staged version of files, while the working directory contains the current versions.

For each tracked file, metadata such as modification time and size is checked. If these differ from the values in the index, the file is considered modified. For better accuracy, the file’s hash can be recomputed and compared with the hash stored in the index.

Then, the same file is compared with its version in the target branch’s tree. If it is both locally modified and different in the target branch, a conflict exists.

In such cases, checkout must be prevented to avoid overwriting the user’s uncommitted changes.

Q 5.3

A detached HEAD state occurs when the HEAD file stores a commit hash directly instead of referencing a branch. This means the user is not currently on any branch.

If a commit is made in this state, a new commit is created but no branch points to it. As a result, it is not part of any branch history and is considered dangling or unreachable.

Although the commit exists in the object store, it may eventually be removed during garbage collection since no reference points to it.

To recover such commits, the user can use the commit hash (for example, from logs) and create a new branch pointing to it, thereby reattaching it to the repository history.

Phase 6:

Q 6.1

Over time, the object store accumulates objects that are no longer reachable from any branch, including old commits, trees, and blobs not referenced in current history.

To clean these up, we perform reachability analysis. Starting from all branch heads in .pes/refs/heads/, we traverse each commit, along with its parent commits, tree objects, and blob objects.

During traversal, we maintain a set of reachable object hashes using a hash set for efficient lookup, allowing us to mark all objects still in use.

After identifying all reachable objects, we scan the entire .pes/objects directory. Any object whose hash is not in the reachable set can be safely deleted.

In a repository with about 100,000 commits and 50 branches, many commits overlap across branches, so typically around 100,000 commits are visited along with their associated objects.

Q 6.2

Running garbage collection at the same time as a commit operation is risky because it can lead to data loss.

For example, during a commit, the system first writes blob and tree objects to the object store but has not yet updated the HEAD or branch reference. At this stage, these new objects are not reachable from any branch.

If garbage collection runs concurrently, it scans for reachable objects starting from branch heads. Since the new objects are not yet referenced, GC may wrongly treat them as unreachable and delete them.